

CAREER AGENT BUILD MANUAL

Pre-Claude implementation guide for Codex + manual setup

Project root: E:\sri\X-Pent-Dev\ClaudeJob

Build goal

Finish the always-on discovery stack and the engineering skeleton for later Claude integration before buying Claude Pro. The system should fetch, normalize, filter, score, store and notify without Claude. Claude is added later only at the intelligence boundary.

Derived from Career Agent V3 - Lean Cloud + Claude Pro Architecture (September 2026)

Implementation details in this manual are concrete engineering choices that expand the V3 design into a step-by-step Windows/GitHub workflow.

How to use this manual

Work through the phases in order. Do not ask Codex to build the whole repository in one giant prompt. Complete a phase, run its verification commands, commit it, then move on.

Operating rule

Code filters. Claude judges. You approve. Before Claude Pro exists, the Claude step is replaced by a mock intelligence provider so the rest of the architecture can be tested end to end.

Phase	Outcome	Claude required?
0	Machine, repo and secrets prepared	No
1	Repository scaffold and profile/evidence files	No
2	Greenhouse, Lever and Ashby acquisition	No
3	Normalization, dedupe, freshness, live checks, filters	No
4	Deterministic scoring and priority buckets	No
5	Persistent state and idempotent pipeline	No
6	Telegram notification/control plane	No
7	GitHub Actions always-on cloud runner	No
8	IntelligenceProvider + MockProvider boundary	No
9	Application artifact skeleton and human-path placeholders	No
10	Gmail/reply loop skeleton	No
11	Pre-Claude acceptance test	No
12	Replace mock provider with Claude later	Yes

Static table of contents

0. Before you start
1. Create the project locally
2. Create the repository structure
3. Create configuration and profile files
4. Create the source adapters
5. Build normalization and filtering
6. Build deterministic scoring
7. Build persistent state
8. Assemble the local pipeline
9. Add Telegram
10. Add GitHub Actions
11. Add the mock intelligence boundary
12. Add application artifact infrastructure

13. Add human-path placeholders
14. Add Gmail closed-loop skeleton
15. Add tests and quality gates
16. Run the full pre-Claude acceptance test
17. What to ask Codex in each phase
18. What NOT to build yet
19. Day-one Claude integration later
20. Troubleshooting and recovery
21. Final checklist and target repository tree

0. Before you start

0.1 What this build should achieve before Claude Pro

- A scheduled GitHub Action can run while your laptop is off.
- The job worker can fetch public jobs from Greenhouse, Lever and Ashby style endpoints/adapters.
- Python can normalize jobs into one schema and remove duplicates.
- Python can reject obvious wrong-location, stale, dead, too-senior and wrong-role jobs conservatively.
- A deterministic score can rank surviving jobs and assign provisional P0/P1/P2/archive buckets.
- Persistent state prevents the same job from being treated as new every run.
- Telegram can notify you about promising candidates.
- A MockProvider can stand in for Claude so downstream approval/application flows are testable.
- The repository contains clean extension points for application, Gmail and interview logic.

0.2 Prerequisites on Windows

Tool	Why	Check command
Git	Source control and GitHub	git --version
Python 3.11+ recommended	Orchestrator and adapters	python --version
PowerShell 7 or Windows PowerShell	Commands in this guide	\$PSVersionTable.PSVersion
VS Code or editor	Manual editing/review	code --version
GitHub account	Private repo + Actions	Manual/browser
Telegram account	Mobile notifications	Manual/mobile

```
git --version
python --version
pip --version
```

Stop condition

If Python or Git is missing, install it first. Do not let Codex hide machine setup problems by generating files you cannot run locally.

0.3 Decide the scope before coding

For the pre-Claude build, use public job data and deterministic rules only. Keep Career Ops integration as an adapter boundary if its exact runtime/API is not yet configured. The V3 source explicitly keeps Career Ops and public ATS sources but removes mandatory paid scraping.

- **Build now:** Greenhouse, Lever, Ashby adapters; normalization; filters; state; Telegram; GitHub Actions; mock intelligence; tests.
- **Reserve now:** Career Ops adapter interface, Claude adapter interface, Gmail interface, application artifact interface.
- **Do later:** Claude judgement, high-quality resume tailoring, personalized outreach, interview reasoning.

1. Create the project locally

1.1 Open PowerShell and create the exact folder

```
E:
cd E:\sri\X-Pent-Dev
mkdir ClaudeJob -Force
cd ClaudeJob
Get-Location
```

Expected location:

```
Path
----
E:\sri\X-Pent-Dev\ClaudeJob
```

1.2 Initialize Git

```
git init
git branch -M main
```

1.3 Create and activate a virtual environment

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

If PowerShell blocks activation, use this only for the current shell:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
```

1.4 Install only the MVP dependencies

```
pip install requests pyyaml python-dotenv pydantic tenacity pytest pytest-cov
pip freeze > requirements.txt
```

Do not install vector databases, embedding models, rerankers, Qwen runtimes, browser automation or Claude SDKs yet.

1.5 Create the base .gitignore

```
.venv/
__pycache__/
.pytest_cache/
.coverage
htmlcov/
.env
*.pyc
*.pyo
*.log
artifacts/generated/
state/runtime/
.DS_Store
Thumbs.db
```

Secret rule

Never commit .env, Telegram bot tokens, Gmail credentials, OAuth refresh tokens or future Claude credentials.

1.6 First commit

```
git add .gitignore requirements.txt
git commit -m "chore: initialize pre-Claude career agent"
```

2. Create the repository structure

2.1 Create directories

```
$dirs = @(
    "orchestrator",
    "sources",
    "profile",
    "telegram",
    "gmail",
    "application",
    "interview",
    "intelligence",
    "state",
    "state\runtime",
    "artifacts",
    "artifacts\generated",
    "tests",
    ".github\workflows"
)
$dirs | ForEach-Object { New-Item -ItemType Directory -Force $_ | Out-Null }
```

2.2 Create placeholder files

```
$files = @(
    "orchestrator\__init__.py",
    "orchestrator\main.py",
    "orchestrator\pipeline.py",
    "orchestrator\policies.py",
    "orchestrator\scorer.py",
    "orchestrator\dedupe.py",
    "orchestrator\freshness.py",
    "orchestrator\live_check.py",
    "sources\__init__.py",
    "sources\career_ops.py",
    "sources\greenhouse.py",
    "sources\lever.py",
    "sources\ashby.py",
    "telegram\__init__.py",
    "telegram\bot.py",
    "telegram\cards.py",
    "gmail\__init__.py",
    "gmail\watcher.py",
    "gmail\classifier.py",
    "application\__init__.py",
    "application\factory.py",
    "application\artifacts.py",
    "interview\__init__.py",
    "interview\prep.py",
    "intelligence\__init__.py",
    "intelligence\provider.py",
    "intelligence\mock.py",
    "intelligence\claude.py",
    "state\store.py",
    "tests\test_pipeline.py",
    "tests\test_scorer.py",
    "tests\test_dedupe.py",
    ".github\workflows\scan_jobs.yml",
    "CLAUDE.md",
    "AGENTS.md",
    "README.md"
)
$files | ForEach-Object { New-Item -ItemType File -Force $_ | Out-Null }
```

2.3 Verify the tree

```
tree /F
```

Why create claude.py now?

The file exists only as a boundary/placeholder. This preserves the V3 upgrade path without making Claude a

current dependency.

2.4 Put project rules in AGENTS.md

Give Codex repository-wide rules so it does not overengineer. Suggested contents:

```
# AGENTS.md

## Goal
Build the pre-Claude Career Agent V3 infrastructure.

### Non-negotiable architecture
- Python handles deterministic work.
- IntelligenceProvider is the only AI boundary.
- MockProvider is the active provider before Claude Pro.
- No embeddings, reranker, local LLM, vector DB, VPS, Selenium or auto-apply.
- No secret values in source control.
- Every source adapter returns the same normalized Job model.
- Every pipeline run must be idempotent.
- Conservative filters: uncertain cases survive for later reasoning.

## Quality
- Add tests for new deterministic logic.
- Keep functions small and typed.
- Log counts at every pipeline stage.
- Do not silently catch exceptions; report source-level failures and continue where safe.
```

3. Create configuration and profile files

3.1 Create .env.example

```
TELEGRAM_BOT_TOKEN=
TELEGRAM_CHAT_ID=

# Optional later
GMAIL_CLIENT_ID=
GMAIL_CLIENT_SECRET=
GMAIL_REFRESH_TOKEN=

# Future Claude integration only
CLAUDE_MODE=mock
```

Copy it locally to .env, then fill only Telegram values when you reach that phase:

```
Copy-Item .env.example .env
```

3.2 Create profile/preferences.yaml

```
role_families:
  - devops
  - cloud
  - platform
  - sre
  - mlops
  - infrastructure

locations:
  preferred:
    - Pune
    - Bengaluru
    - Remote India
  allowed_country:
    - India

seniority:
  min_years: 0
  max_years: 3
  reject_titles:
    - Senior
    - Staff
    - Principal
    - Lead
    - Manager

freshness:
  max_age_hours: 168
  strong_age_hours: 24

keywords:
  strong:
    - Python
    - Linux
    - Docker
    - Kubernetes
    - Git
    - CI/CD
  useful:
    - Terraform
    - AWS
    - GCP
    - Azure
    - Ansible
    - Ceph

scoring:
  p0: 85
  p1: 80
  p2: 65
```

Treat this as a starting configuration, not a statement of your actual skills. Replace values with your real preferences.

3.3 Create profile/master_profile.yaml

```
name: Srinivas Sivakumar
headline: ""
location: ""
experience_years: null

skills: []
projects: []
experience: []
education: []
certifications: []
```

Populate this manually from your real resume/profile. Do not let Codex invent missing experience.

3.4 Create profile/evidence_bank.yaml

```
evidence:
- id: example_project
  type: project
  title: "Replace with a real project"
  claims:
    - "Replace with a verified statement"
  technologies: []
  metrics: []
  source: "resume/project notes"
  verified: false
```

Grounding rule from V3

The evidence bank is the truth boundary. Future Claude may rephrase, reorder, shorten and emphasize, but must not invent skills, metrics, outcomes, scale or experience.

3.5 Create profile/voice_dna.md

```
# Voice DNA

## Tone
- Direct
- Technical but readable
- No fake enthusiasm
- No inflated claims

## Avoid
- "rockstar"
- "guru"
- invented metrics
- generic AI-sounding filler

## Preferred outreach length
- LinkedIn: 2-4 short sentences
- Recruiter email: under 150 words
- Referral message: under 120 words
```

3.6 Commit the profile skeleton

```
git add profile .env.example AGENTS.md
git commit -m "feat: add profile evidence and preferences skeleton"
```

4. Create the source adapters

4.1 Define one normalized Job model first

Create a typed model in orchestrator/models.py. Add this file even though it was not in the original tree; it prevents each adapter from inventing its own schema.

```
from datetime import datetime
from pydantic import BaseModel, HttpUrl

class Job(BaseModel):
    source: str
    source_job_id: str
    company: str
    title: str
    location: str | None = None
    description: str = ""
    url: str
    posted_at: datetime | None = None
    fetched_at: datetime
    employment_type: str | None = None
    department: str | None = None
    raw: dict = {}

    @property
    def canonical_key(self) -> str:
        return f"{self.source}:{self.source_job_id}"
```

4.2 Greenhouse adapter

The adapter should accept a configurable list of company board identifiers, fetch each board, and return Job objects. Keep HTTP details inside sources/greenhouse.py; the pipeline should only see normalized jobs.

```
class GreenhouseSource:
    name = "greenhouse"

    def __init__(self, board_tokens: list[str]):
        self.board_tokens = board_tokens

    def fetch(self) -> list[Job]:
        jobs: list[Job] = []
        for token in self.board_tokens:
            # GET public board endpoint
            # validate response
            # map every record to Job
            # preserve raw payload in raw={...}
            ...
        return jobs
```

4.3 Lever adapter

Implement the same public interface: constructor/configuration plus fetch() → list[Job]. Do not leak Lever-specific field names into the rest of the codebase.

4.4 Ashby adapter

Follow the exact same contract. If a company-specific public endpoint changes, only this adapter should require repair.

4.5 Career Ops adapter boundary

Because the V3 source says to keep Career Ops, reserve sources/career_ops.py now. If you have not installed/configured its exact interface, make the adapter return an empty list and log that it is disabled instead of inventing an API.

```
class CareerOpsSource:
    name = "career_ops"

    def __init__(self, enabled: bool = False):
        self.enabled = enabled

    def fetch(self) -> list[Job]:
        if not self.enabled:
            return []
        raise NotImplementedError("Configure the real Career Ops integration first")
```

4.6 Add source configuration

```
# config/sources.yaml
sources:
  greenhouse:
    enabled: true
    boards: []
  lever:
    enabled: true
    companies: []
  ashby:
    enabled: true
    companies: []
  career_ops:
    enabled: false
```

Add real target companies gradually. Start with 5-10 boards, not hundreds. This makes parser debugging much easier.

4.7 Source-level resilience

- Use a reasonable timeout for every HTTP request.
- Use retries only for transient failures such as timeouts/5xx responses.
- Log source name, company identifier and error.
- One failed company should not crash all other sources.
- Do not retry permanent 404/invalid configuration forever.
- Never treat an empty response as proof that all jobs are gone without recording the source status.

4.8 Manual test for adapters

```
python -m orchestrator.main --mode fetch-only --limit 20
```

Expected output pattern:

```
greenhouse: fetched 12
lever: fetched 8
ashby: fetched 5
career_ops: disabled
total_raw_jobs: 25
```

5. Build normalization and filtering

5.1 Pipeline order

```
fetch
-> normalize
-> dedupe
-> live check
-> freshness
-> location
-> seniority
-> role-family
-> deterministic pre-score
-> store candidates
```

Important

The V3 design says cheap filters must not permanently reject a potentially excellent opportunity unless the rule is conservative. Prefer "keep uncertain" over aggressive rejection.

5.2 Normalization rules

- Trim whitespace and normalize repeated spaces.
- Keep original title and company text for display.
- Create lowercase helper values only for matching.
- Convert posted times to UTC internally when possible.
- Preserve raw source payload for debugging.
- Never fabricate posted_at if the source does not supply one; use null and mark freshness as unknown.

5.3 Dedupe

Use multiple levels. First source_job_id within a source, then URL normalization, then a conservative cross-source fingerprint for obvious duplicates.

```
def fingerprint(job: Job) -> str:
    company = normalize_text(job.company)
    title = normalize_text(job.title)
    location = normalize_text(job.location or "")
    return f"{company}|{title}|{location}"
```

Do not fuzzy-merge jobs merely because titles are similar. "DevOps Engineer" and "Senior DevOps Engineer" may be different openings.

5.4 Freshness

```
def freshness_status(job, now, max_age_hours=168):
    if job.posted_at is None:
        return "unknown"
    age_h = (now - job.posted_at).total_seconds() / 3600
    if age_h < 0:
        return "unknown"
    if age_h <= 24:
        return "fresh"
    if age_h <= max_age_hours:
        return "acceptable"
    return "stale"
```

Unknown freshness should usually survive with fewer score points rather than be automatically deleted.

5.5 Live check

For shortlisted jobs, verify that the target URL still resolves. Avoid expensive live checks for every raw record if the source itself is authoritative. A 404/410 is a strong dead-listing signal; timeouts are not.

5.6 Location filter

Make location logic configurable and conservative. Examples that should usually survive: "India - Remote", "Pune / Hybrid", "Bengaluru", and unknown remote status if the company otherwise fits. Reject only clearly incompatible countries/onsite requirements based on your preferences.

5.7 Seniority filter

Use obvious title terms first. Then parse explicit experience requirements if available. Do not reject on ambiguous words alone.

```
HARD_SENIOR_TERMS = {  
    "staff", "principal", "director", "head of", "vp", "vice president"  
}  
SOFT_SENIOR_TERMS = {"senior", "lead", "manager"}
```

Hard terms can be rejected according to your preferences. Soft terms can be marked uncertain if the JD itself appears junior-friendly.

5.8 Role-family filter

Create allow/deny signals around your target roles. Include title and a small amount of JD text. Keep ambiguous platform/cloud roles rather than filtering only by exact titles.

6. Build deterministic scoring

6.1 Split objective and subjective scoring

Before Claude, score only signals Python can justify reliably. The V3 conceptual 100-point model includes evidence fit, freshness, seniority, location, human path, posting quality and application friction; subjective judgement will later move to Claude.

Signal	Pre-Claude points	How
Freshness	0-15	Posted age / unknown freshness
Seniority realism	0-15	Title + explicit experience requirements
Location	0-10	Preferred/allowed location rules
Keyword/evidence overlap	0-25	Verified deterministic skill/evidence matches
Posting quality	0-5	Description completeness, clear responsibilities
Application friction	0-5	Simple/direct application signals if known
Human path	0-5 initially	Existing known contact data only; no expensive research
Reserved subjective capacity	0-20	Do not fake it. Mock provider can simulate later judgement.

6.2 Provisional buckets before Claude

Because 20 points are reserved for later reasoning, do not pretend the deterministic pre-score is the final V3 score. Use provisional labels such as strong_candidate / review / weak rather than claiming final P0/P1 unless you explicitly define them as provisional.

```
if pre_score >= 68:
    bucket = "strong_candidate"
elif pre_score >= 55:
    bucket = "review"
else:
    bucket = "weak"
```

When Claude is added, the final thresholds can return to the V3 concept: archive below 65, P2 65-79, P1 80-84, P0 85+.

6.3 Evidence matching

Load only verified entries from evidence_bank.yaml. Match technologies/keywords deterministically. Record WHICH evidence matched so the later Claude payload is grounded.

```
matched_evidence = [
    {
        "evidence_id": "ncbs_ceph",
        "matched_terms": ["linux", "docker", "storage"]
    }
]
```

6.4 Score explanation object

Every score should be explainable and stored. Do not store only a number.

```
{
  "pre_score": 63,
  "signals": {
    "freshness": 15,
    "seniority": 15,
    "location": 10,
    "evidence_overlap": 18,
    "posting_quality": 3,
    "application_friction": 2
  },
  "matched_evidence_ids": ["ncbs_ceph"],
  "warnings": ["terraform gap", "freshness unknown"]
}
```

7. Build persistent state

7.1 Start simple

The V3 guide allows JSONL or a carefully serialized SQLite file in a private GitHub repository initially. For easiest GitHub Actions operation, use append-friendly JSONL plus a compact index/state JSON until concurrency actually becomes a problem.

7.2 Files

```
state/  
  jobs.jsonl  
  applications.jsonl  
  contacts.jsonl  
  events.jsonl  
  runtime/  
    seen_jobs.json  
    last_run.json
```

7.3 State invariants

- A canonical job key is stable across runs.
- Seen jobs are not re-notified as new unless their state meaningfully changes.
- A failed source does not mark its previously known jobs dead.
- Every important transition appends an event.
- Application status changes are explicit and timestamped.
- One GitHub workflow should own state writes initially to avoid concurrent mutation.

7.4 Example event

```
{  
  "event_id": "...",  
  "type": "job_discovered",  
  "job_key": "greenhouse:12345",  
  "timestamp": "2026-09-08T05:00:00Z",  
  "payload": {"pre_score": 68}  
}
```

7.5 Git state strategy

For the MVP, allow the scheduled workflow to commit state changes back to the private repository. Use one serialized scan workflow. Later, if concurrency or write conflicts become real, move state to a transactional cloud store. Do not solve that hypothetical problem now.

8. Assemble the local pipeline

8.1 orchestrator/pipeline.py responsibilities

```
def run_pipeline(config, store, notifier=None, intelligence=None):
    raw = fetch_all_sources(config)
    normalized = normalize_jobs(raw)
    unique = dedupe_jobs(normalized)
    filtered = apply_conservative_filters(unique, config)
    scored = score_jobs(filtered, config)
    new_candidates = store.diff_new_or_changed(scored)

    if intelligence is not None:
        evaluated = intelligence.evaluate_jobs(new_candidates, load_evidence())
    else:
        evaluated = new_candidates

    store.persist(evaluated)

    if notifier is not None:
        notifier.send_candidates(evaluated)

    return build_run_summary(...)
```

8.2 orchestrator/main.py CLI

Make local debugging easy with explicit modes.

```
python -m orchestrator.main --mode fetch-only
python -m orchestrator.main --mode scan --dry-run
python -m orchestrator.main --mode scan
python -m orchestrator.main --mode notify-test
```

8.3 Logging counts

```
raw=153
normalized=153
unique=121
live_or_unknown=118
fresh_or_unknown=72
location_ok_or_unknown=33
seniority_ok_or_unknown=19
role_family_ok_or_unknown=13
strong_or_review=5
new_or_changed=2
```

This count trail is critical

The whole V3 budget strategy depends on proving that ordinary code reduces a large discovery set to a tiny set of serious or uncertain jobs.

8.4 Dry-run behavior

A dry run should fetch and score but must not commit state, send Telegram messages, generate artifacts or modify applications. Use it heavily while tuning filters.

9. Add Telegram

9.1 Create a Telegram bot manually

1. Open Telegram and start a chat with BotFather.
2. Create a new bot with /newbot.
3. Choose a display name and a unique bot username.
4. Copy the bot token. Put it in your local .env as TELEGRAM_BOT_TOKEN.
5. Send at least one message to your new bot from your own Telegram account.
6. Determine your chat ID using the bot update mechanism or a small local helper. Store it as TELEGRAM_CHAT_ID.

Do not commit the token

The token belongs in .env locally and GitHub Actions Secrets in the cloud.

9.2 Create telegram/bot.py

At minimum, implement send_message(text) and a health/test method. Use plain HTTPS requests if you want to avoid another dependency.

9.3 Candidate card format

```
STRONG CANDIDATE - 68 deterministic points
Red Hat
Associate DevOps Engineer
Pune / Hybrid
Posted: 4 hours ago

Evidence overlap:
- Linux
- Docker
- Python

Warnings:
- Terraform gap

Status: awaiting later intelligence / human review

[VIEW JOB] [PREPARE] [SKIP]
```

Before Claude exists, be transparent that the score is deterministic/provisional. Do not label a job as a final Claude P0.

9.4 Buttons

Use inline buttons for VIEW JOB, PREPARE and SKIP. PREPARE can initially create an application task/event and a placeholder artifact folder; it does not need Claude yet.

9.5 Telegram test

```
python -m orchestrator.main --mode notify-test
```

Pass condition: the message arrives on your phone and contains no secret values or malformed URLs.

10. Add GitHub Actions

10.1 Create a private GitHub repository

1. On GitHub, create a new PRIVATE repository named ClaudeJob or career-agent.
2. Do not initialize it with a README if your local repository already has one.
3. Copy the repository URL.
4. Add it as origin and push main.

```
git remote add origin https://github.com/YOUR_USERNAME/ClaudeJob.git
git push -u origin main
```

10.2 Add GitHub Actions secrets

In the repository: Settings → Secrets and variables → Actions. Add:

Secret	Value
TELEGRAM_BOT_TOKEN	Your BotFather token
TELEGRAM_CHAT_ID	Your private chat ID

10.3 Minimal scan_jobs.yml

```
name: Scan jobs

on:
  workflow_dispatch:
  schedule:
    - cron: "30 23 * * *"
    - cron: "30 2 * * *"
    - cron: "30 5 * * *"
    - cron: "30 8 * * *"
    - cron: "30 11 * * *"
    - cron: "30 14 * * *"
    - cron: "30 17 * * *"
    - cron: "30 20 * * *"

permissions:
  contents: write

jobs:
  scan:
    runs-on: ubuntu-latest
    concurrency:
      group: career-agent-state-writer
      cancel-in-progress: false
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - run: pip install -r requirements.txt

      - name: Run scanner
        env:
          TELEGRAM_BOT_TOKEN: ${ secrets.TELEGRAM_BOT_TOKEN }
          TELEGRAM_CHAT_ID: ${ secrets.TELEGRAM_CHAT_ID }
        run: python -m orchestrator.main --mode scan

      - name: Persist state
        run: |
          git config user.name "career-agent-bot"
          git config user.email "career-agent-bot@users.noreply.github.com"
          git add state/
          git diff --cached --quiet || git commit -m "state: update career agent"
          git push
```

The example cron values are UTC. Adjust them to the scan times you actually want. The V3 document gives eight scans per day as an example, not a mandatory schedule.

10.4 First cloud run

1. Commit and push scan_jobs.yml.
2. Open the Actions tab.
3. Select Scan jobs.
4. Run workflow manually.
5. Open logs and verify stage counts.
6. Confirm a Telegram notification if a test candidate is emitted.
7. Confirm state files changed only as expected.

10.5 Laptop-off test

R1 acceptance criterion

Once a scheduled run can fetch fresh jobs, score them and notify your phone while the laptop is completely

off, the V3 R1 objective is achieved.

11. Add the mock intelligence boundary

11.1 Define IntelligenceProvider

```
from abc import ABC, abstractmethod

class IntelligenceProvider(ABC):
    @abstractmethod
    def evaluate_jobs(self, jobs, evidence): ...

    @abstractmethod
    def tailor_application(self, job, evidence): ...

    @abstractmethod
    def classify_reply(self, message): ...

    @abstractmethod
    def prepare_interview(self, application): ...
```

11.2 Implement MockProvider

The mock should be deterministic. Its purpose is integration testing, not pretending to be intelligent.

```
class MockProvider(IntelligenceProvider):
    def evaluate_jobs(self, jobs, evidence):
        output = []
        for job in jobs:
            output.append({
                "job": job,
                "verdict": "mock_review",
                "priority": "P1" if job.pre_score >= 65 else "P2",
                "reason": "Mock provider - replace with Claude later"
            })
        return output

    def tailor_application(self, job, evidence):
        return {"status": "mock", "files": []}

    def classify_reply(self, message):
        return {"type": "unknown", "confidence": 0.0}

    def prepare_interview(self, application):
        return {"status": "mock"}
```

11.3 Provider factory

```
def get_intelligence_provider(mode: str):
    if mode == "mock":
        return MockProvider()
    if mode == "claude":
        raise RuntimeError("Claude provider not configured yet")
    raise ValueError(f"Unknown intelligence mode: {mode}")
```

11.4 Candidate payload builder

Build the exact structured payload now so later Claude integration is small. It should include the job, deterministic pre-analysis and only relevant verified evidence.

```
{
  "job_id": "J431",
  "title": "MLOps Engineer",
  "company": "Example",
  "location": "Pune / Hybrid",
  "posted_age_hours": 5,
  "pre_analysis": {
    "python_docker_linux": "strong",
    "kubernetes": "moderate",
    "terraform": "gap",
    "seniority": "realistic"
  },
  "verified_evidence": ["ncbs_ceph", "linkedin_assistant"],
  "jd": "...
}
```

Value

When Claude Pro is purchased, the rest of the repository should not change. Only the provider implementation and provider configuration should change.

12. Add application artifact infrastructure

12.1 Application factory behavior before Claude

PREPARE should create a durable application record and artifact directory even if the content is placeholder/template based.

```
artifacts/generated/
red-hat/
  associate-devops-engineer/
    job.json
    fit_report.json
    evidence_matrix.md
    resume.md
    recruiter_email.txt
    referral_message.txt
    linkedin_message.txt
```

12.2 Do not fake generated content

Before Claude, files can contain a clear placeholder such as "Pending Claude tailoring" plus deterministic evidence/requirements extracted by code. Do not generate polished claims that appear AI-vetted.

12.3 application/factory.py flow

```
def prepare_application(job, evaluation, evidence, store, provider):
    application = store.create_application(job, status="preparing")
    paths = create_artifact_directory(job)
    write_job_json(paths, job)
    write_fit_report(paths, evaluation)
    write_deterministic_evidence_matrix(paths, job, evidence)

    generated = provider.tailor_application(job, evidence)
    write_provider_outputs(paths, generated)

    store.update_application(application.id, status="prepared")
    return paths
```

12.4 Human approval invariant

No auto-apply

The V3 architecture puts the application factory behind P0/P1 plus human approval. Before Claude, preserve the same gate. PREPARE can build artifacts; applying or sending remains manual/approved.

13. Add human-path placeholders

13.1 Scope before Claude

Do not build logged-in LinkedIn scraping or browser bots. Store manually discovered/public contact URLs and prepare the data model. This preserves the V3 human-controlled LinkedIn boundary.

```
{
  "job_key": "greenhouse:12345",
  "people": [
    {
      "type": "recruiter",
      "name": "",
      "linkedin_url": "",
      "source": "manual",
      "verified": false
    }
  ]
}
```

13.2 Priority fields

- Recruiter
- Hiring manager
- Existing connection
- Employee/alumni
- Future teammate
- Founder when appropriate

These priority categories mirror the V3 strategy, but the pre-Claude build only needs storage and UI hooks.

13.3 Telegram link behavior

If a LinkedIn URL is known, expose an OPEN LINKEDIN button. The user opens the profile and sends manually. Do not implement auto-connect or auto-DM.

14. Add Gmail closed-loop skeleton

14.1 Keep this optional until R1 is stable

Gmail is valuable, but do not delay the always-on discovery milestone for it. Build the interface after job discovery, state and Telegram work reliably.

14.2 Event types to support

```
application_receipt
recruiter_reply
assessment
interview_invite
rejection
offer
unknown
```

14.3 Deterministic first-pass classifier

Classify obvious messages using sender, subject, known application/company, thread and keywords. Ambiguous messages remain unknown until future Claude classification.

```
def classify_message(msg, known_apps):
    text = f"{msg.subject} {msg.snippet}".lower()

    if "thank you for applying" in text:
        return "application_receipt"
    if "interview" in text or "schedule a call" in text:
        return "interview_invite"
    if "assessment" in text or "coding test" in text:
        return "assessment"
    if "unfortunately" in text and "application" in text:
        return "rejection"
    return "unknown"
```

14.4 Safety/actions

Type	Pre-Claude action
application_receipt	Record event; no urgent notification
recruiter_reply	Urgent Telegram notification; pause follow-up
assessment	Record link/deadline; urgent notification
interview_invite	Move stage; urgent notification; create interview task
rejection	Record outcome; cancel follow-up
offer	Immediate human review; never auto-reply
unknown	Store for manual/future Claude classification

14.5 Gmail credential caution

Use OAuth credentials with the minimum required scope. Store secrets outside Git. If Gmail setup becomes time-consuming, leave watcher.py behind a disabled feature flag until the core system is working.

15. Add tests and quality gates

15.1 Minimum unit tests

File	Must test
test_dedupe.py	Same source ID, URL duplicates, non-duplicates
test_scorer.py	Freshness, location, seniority, evidence points
test_pipeline.py	Stage ordering, dry-run, failed source isolation
test_state.py	Idempotency, new/changed detection, event append
test_policies.py	Conservative rejection and uncertain survival

15.2 Run tests locally

```
pytest -q
pytest --cov=orchestrator --cov=sources --cov=state --cov-report=term-missing
```

15.3 Add a CI workflow

Create `.github/workflows/test.yml` and run it on pushes/pull requests. Keep scheduled scanning separate from tests/state writes.

15.4 Test fixtures

Use saved synthetic/sample job payloads. Do not make every unit test hit live ATS endpoints; that makes tests flaky and can hide parser regressions.

15.5 Golden acceptance fixture

Create a small fixture set containing: one perfect junior role, one senior role, one wrong-country role, one duplicate, one stale role, one unknown-date role and one borderline role. The expected pipeline result should be stable.

16. Run the full pre-Claude acceptance test

16.1 Local acceptance

1. Activate .venv.
2. Run `pytest -q` and get a clean pass.
3. Run a fetch-only scan and inspect raw job counts.
4. Run a dry scan and inspect stage counts and explanations.
5. Run a real local scan with MockProvider.
6. Confirm state changes are idempotent by running the same scan again.
7. Confirm only new/changed strong candidates are notified.
8. Tap PREPARE and verify the artifact folder and application record are created.

16.2 Cloud acceptance

1. Push a clean commit to main.
2. Run GitHub Actions manually.
3. Verify successful source fetching and pipeline counts.
4. Verify state commit/push succeeds.
5. Verify Telegram delivery.
6. Turn the laptop off before a scheduled run.
7. Confirm the scheduled run still executes and your phone still receives a qualifying notification.

16.3 Definition of done before paying for Claude

Buy Claude only after this

GitHub Actions + sources + Python filtering/scoring + state + Telegram + mock provider work end to end. At that point the \$20 Claude subscription is used to improve judgement and application quality, not to debug infrastructure.

17. What to ask Codex in each phase

Use narrow prompts. After each prompt, review the diff, run tests and commit. Suggested prompts:

17.1 Scaffold prompt

```
We are building Career Agent V3 inside E:\sri\X-Pent-Dev\ClaudeJob.
Read AGENTS.md first.
Create the repository scaffold only. Do not implement business logic yet.
Add __init__.py files, config loading, typed Job model, .env.example and README setup instructions.
Do not add AI SDKs, vector DBs, embeddings, Selenium or cloud services.
Run tests/import checks and summarize exactly what changed.
```

17.2 Source adapter prompt

```
Implement public job source adapters for Greenhouse, Lever and Ashby behind a common fetch() ->
list[Job] contract.
Keep source-specific fields inside each adapter.
Add timeout/retry/error isolation and fixture-based tests.
Do not change scoring or Telegram code.
Do not invent a Career Ops API; leave that adapter disabled unless its real interface is present.
```

17.3 Filtering prompt

```
Implement normalization, dedupe, freshness, live-check, location, seniority and role-family
filtering.
Rules must be conservative: uncertain jobs survive with warnings.
Every rejected job must have a machine-readable reason.
Add tests for obvious rejects and borderline survivors.
```

17.4 Scoring prompt

```
Implement deterministic pre-scoring only.
Use objective signals from preferences.yaml and verified evidence_bank entries.
Return both a score and a breakdown/explanation.
Do not claim the result is a final Claude P0/P1 judgement.
Add unit tests for each scoring component.
```

17.5 State prompt

```
Implement append-friendly JSONL state plus a small seen-job index.
The pipeline must be idempotent.
A second identical run must not notify the same job as new.
Record events for discovery, notification, prepare and skip.
Use one writer model suitable for a serialized GitHub Actions workflow.
```

17.6 Telegram prompt

```
Implement Telegram notifications using TELEGRAM_BOT_TOKEN and TELEGRAM_CHAT_ID from environment
variables.
Add VIEW JOB, PREPARE and SKIP buttons.
Before Claude exists, label scores as deterministic/provisional.
Never log secrets.
Add a notify-test CLI mode.
```

17.7 GitHub Actions prompt

```
Create GitHub Actions workflows for tests and scheduled scanning.
Scheduled scanning must use one serialized state writer, install requirements, run the scanner, and
commit only state changes.
Use GitHub Secrets for Telegram values.
Do not add paid cloud infrastructure.
```

17.8 Intelligence boundary prompt

```
Implement IntelligenceProvider plus MockProvider.  
The mock must be deterministic and clearly marked as mock output.  
Create a provider factory controlled by CLAUDE_MODE=mock.  
Create claude.py only as a non-functional placeholder with a clear error until configured.  
Do not install an Anthropic SDK yet.
```

17.9 Application skeleton prompt

```
Implement the application factory and artifact directory creation behind the PREPARE action.  
Write job.json, fit_report.json and deterministic evidence_matrix.md.  
Provider-generated files must show explicit placeholder content when MockProvider is active.  
No auto-apply, auto-email or LinkedIn automation.
```

18. What NOT to build yet

Do not build	Reason
BGE embeddings	Only add if keyword/evidence matching later sends too many jobs to Claude.
Reranker	Only add if embeddings still produce too many false positives.
Qwen/local LLM	Only add if Claude capacity is being wasted on routine JD extraction.
Vector database	No demonstrated need in the MVP.
VPS/AWS server	GitHub Actions is the planned always-on worker.
Paid Apify dependency	V3 explicitly makes paid scraping optional later.
Logged-in LinkedIn scraper	V3 keeps LinkedIn human-controlled.
Auto-connect/auto-DM	Manual boundary and account risk.
Auto-apply	Human approval gate first.
Complex distributed state	Single serialized workflow is enough initially.

Anti-overengineering rule

If Claude Pro later lasts comfortably and quality is good, do not add extra AI layers simply because they are technically interesting.

19. Day-one Claude integration later

19.1 What should already be complete

- Candidate payload builder
- Evidence bank
- IntelligenceProvider interface
- MockProvider
- Application factory
- Telegram approval flow
- State and usage/event logging

19.2 Replace only the provider

```
CLAUDE_MODE=cclaude
```

Then implement intelligence/claude.py so it satisfies the existing interface. The first method to make real should be `evaluate_jobs()`. Application tailoring comes after ranking quality is trustworthy.

19.3 Claude job-evaluation request

Send only 3-5 serious/uncertain candidates, not every discovered job. Include exact JD, deterministic pre-analysis, and relevant verified evidence. Ask for realistic fit, critical gaps, transferable evidence, final priority and whether the opportunity deserves application-generation capacity.

19.4 Usage governor

Usage state	Behavior
0-50%	Normal: evaluate strongest/uncertain candidates
50-70%	Conservative: reduce marginal evaluations
70-80%	Selective: P0-like opportunities only
80%+	Reserve: protect capacity for interviews, assessments, recruiter replies and exceptional opportunities

This directly reflects the V3 hierarchy: interviews and assessments outrank routine job evaluation.

19.5 Claude integration acceptance

1. Run the same candidate fixture through MockProvider and ClaudeProvider.
2. Confirm Claude output is valid structured data and grounded only in provided evidence.
3. Confirm final P0/P1/P2/archive thresholds are applied consistently.
4. Confirm P2/archive jobs do not trigger resume/company/recruiter deep work.
5. Confirm application generation only starts after your approval.

20. Troubleshooting and recovery

20.1 GitHub Action fetches zero jobs

- Run the same adapter locally with debug logging.
- Check whether company board identifiers are correct.
- Inspect HTTP status and response shape.
- Do not rewrite all adapters because one source changed.
- Keep source failures isolated and visible in the run summary.

20.2 Too many candidates survive

- Inspect stage counts.
- Strengthen deterministic filters only where false positives are obvious.
- Tune role-family keywords and seniority rules.
- Do not add embeddings yet unless simple rules are demonstrably inadequate.

20.3 Good jobs are being rejected

- Review rejection reasons from stored events.
- Change hard rejection to warning/uncertain for ambiguous signals.
- Prefer a slightly larger Claude candidate set over irreversible false negatives.

20.4 Duplicate Telegram notifications

- Check canonical key stability.
- Check whether state was committed successfully after the previous workflow.
- Check whether URL query strings are changing the fingerprint.
- Ensure notifications use new_or_changed jobs, not all strong jobs.

20.5 State push conflicts

- Confirm concurrency group is enabled and cancel-in-progress is false.
- Avoid multiple workflows writing the same state files.
- Use git pull --rebase before push only if necessary and carefully tested.
- Move to transactional cloud state only when conflicts become persistent and real.

20.6 Telegram does not send

- Confirm the bot received a message from you first.
- Check chat ID and token locally without printing the token.
- Check GitHub Secrets names exactly match the workflow.
- Use the notify-test mode before debugging the entire pipeline.

20.7 Codex starts overengineering

Point it back to AGENTS.md and reject unrelated architecture changes. Ask it to solve only the failing acceptance criterion. The V3 design intentionally delays OSS AI, rerankers, paid storage and extra dashboards until data proves a need.

21. Final checklist and target repository tree

21.1 Pre-Claude checklist

- [] Project lives at E:\sri\X-Pent-Dev\ClaudeJob
- [] Git repository initialized and private GitHub remote configured
- [] .venv and requirements are reproducible
- [] .env is ignored; .env.example is committed
- [] master_profile, evidence_bank, preferences and voice_dna exist
- [] Greenhouse adapter works
- [] Lever adapter works
- [] Ashby adapter works
- [] Career Ops boundary exists without invented API behavior
- [] Normalization works
- [] Dedupe works
- [] Freshness/live checks work conservatively
- [] Location and seniority filtering are configurable
- [] Deterministic score has an explanation breakdown
- [] State is idempotent
- [] Telegram notify-test works
- [] GitHub Actions manual run works
- [] Scheduled laptop-off run works
- [] MockProvider is the active intelligence provider
- [] PREPARE creates application state/artifact skeleton
- [] No auto-apply or automated LinkedIn sending exists
- [] Tests pass locally and in GitHub Actions

21.2 Target tree

```

E:\sri\X-Pent-Dev\ClaudeJob\
|
+-- orchestrator\
|   +-- __init__.py
|   +-- main.py
|   +-- models.py
|   +-- pipeline.py
|   +-- policies.py
|   +-- scorer.py
|   +-- dedupe.py
|   +-- freshness.py
|   +-- live_check.py
|
+-- sources\
|   +-- career_ops.py
|   +-- greenhouse.py
|   +-- lever.py
|   +-- ashby.py
|
+-- profile\
|   +-- master_profile.yaml
|   +-- evidence_bank.yaml
|   +-- preferences.yaml
|   +-- voice_dna.md
|
+-- intelligence\
|   +-- provider.py
|   +-- mock.py
|   +-- claude.py
|
+-- telegram\
|   +-- bot.py
|   +-- cards.py
|
+-- gmail\
|   +-- watcher.py
|   +-- classifier.py
|
+-- application\
|   +-- factory.py
|   +-- artifacts.py
|
+-- interview\
|   +-- prep.py
|
+-- state\
|   +-- store.py
|   +-- jobs.jsonl
|   +-- applications.jsonl
|   +-- contacts.jsonl
|   +-- events.jsonl
|   +-- runtime\
|
+-- artifacts\
|   +-- generated\
|
+-- tests\
|
+-- .github\workflows\
|   +-- scan_jobs.yml
|   +-- test.yml
|
+-- .env.example
+-- .gitignore
+-- requirements.txt
+-- AGENTS.md
+-- CLAUDE.md
+-- README.md

```

21.3 The moment to buy Claude Pro

Ready

Buy the subscription when this entire deterministic pipeline is stable enough that switching `CLAUDE_MODE` from mock to claude is the main remaining intelligence step. From then on, spend Claude capacity on judgement, strong applications, recruiter responses and interview preparation - not fetching, dedupe or scheduling.

End of manual